

Experiment No. 2:

Aim: To implement the Stack data structure using an array in C and perform basic stack operations such as Push, Pop, Peek, and Display.

Tools:

1. Programming Language: C
2. IDE: Visual Studio Code
3. Compiler: GCC
4. Operating System: Windows 10/11

Theory:

A stack is a linear data structure that follows the **Last In First Out (LIFO)** principle. The last element inserted into the stack is the first one to be removed. Stack operations are performed only at one end called the **Top**. The primary operations of a stack are **Push, Pop, Peek, and Display**. Push inserts a new element onto the stack, while Pop removes the top element. Peek returns the top element without removing it. An array can be used to implement a stack by maintaining a variable called **top** that stores the index of the last inserted element. Overflow occurs when attempting to push an element into a full stack, whereas Underflow occurs when attempting to pop from an empty stack. Stacks are widely used in function calls, expression evaluation, undo operations, and browser history.

Algorithm:

Push Operation

1. Start.
2. Check if the stack is full ($\text{top} == N-1$).
3. If full, display "Overflow".
4. Otherwise increment top.
5. Insert the element at $\text{stack}[\text{top}]$.
6. Stop.

Pop Operation

1. Start.

2. Check if the stack is empty ($\text{top} == -1$).
3. If empty, display "Underflow".
4. Otherwise print the top element.
5. Decrement top.
6. Stop.

Peek Operation

1. Check whether the stack is empty.
2. If empty, display "Stack Empty".
3. Otherwise display the top element.

Display Operation

1. Check whether the stack is empty.
2. If empty, display "Stack Empty".
3. Otherwise print all elements from top to 0.

Program:

```
#include <stdio.h>

#define N 5

int stack[N];

int top=-1;

void push(){
    int x;

    printf("Enter the number you want to insert: ");

    scanf("%d",&x);

    if(top==N-1){
        printf("Overflow \n");
```

```

    }else{
        top++;
        stack[top]=x;
    }
}

void pop(){
    if(top== -1){
        printf("The stack is empty \n");
    }else{
        printf("Removed %d \n",stack[top]);
        top--;
    }
}

void display(){
    if (top== -1){
        printf("Underflow\n");
    }else{

        for(int i=top;i>=0;i--){
            printf("%d \n",stack[i]);
        }
    }
}

void peek(){
    if(top== -1){
        printf("Empty Stack\n");
    }
}

```

```
}  
else{  
    printf("The top element is: %d \n",stack[top]);  
}  
}  
  
int main() {  
  
    int choice;  
    do{  
        printf("\nPush-1;Pop-2;Display-3;Peek-4; Break-5\nEnter your choice: ");  
        scanf("%d",&choice);  
  
        switch (choice){  
            case 1: push();break;  
            case 2: pop(); break;  
            case 3: display(); break;  
            case 4: peek();break;  
            case 5: printf("Exiting.");break;  
            default: printf("Invalid choice\n");}  
  
    } while(choice!=5);  
  
    return 0;  
}
```

Output:

Output

```
Push-1;Pop-2;Display-3;Peek-4; Break-5
Enter your choice: 1
Enter the number you want to insert: 20
```

```
Push-1;Pop-2;Display-3;Peek-4; Break-5
Enter your choice: 2
Removed 20
```

```
Push-1;Pop-2;Display-3;Peek-4; Break-5
Enter your choice: 3
Underflow
```

```
Push-1;Pop-2;Display-3;Peek-4; Break-5
Enter your choice: 4
Empty Stack
```

```
Push-1;Pop-2;Display-3;Peek-4; Break-5
Enter your choice: |
```

Learning Outcome:

LO2.1 Implement Push, Pop and Peek.

LO2.2 Analyse stack operations.

Course Outcome:

After completion of the course the learner should be able to:

CO1: Differentiate linear and non-linear data structures based on their organization, operations, and

. computational efficiency.

CO2: Demonstrate the use of stacks and queues to solve real-world computational problems.

CO3: Implement and analyse linked list structures and their operations for dynamic data management.

CO4: Make use of tree structures for hierarchical data representation and efficient searching.

CO5: Examine real-world problems using graph data structures and associated techniques.

CO6: Evaluate advanced data structures based on performance characteristics and application

requirements.

Conclusion:

The experiment was successfully performed. A stack was implemented using an array in C. All basic operations including Push, Pop, Peek, and Display were executed correctly. The experiment demonstrated the LIFO principle and helped in understanding stack implementation along with Overflow and Underflow conditions.

Viva Questions:

Q1. What is a stack?

A stack is a linear data structure that follows the LIFO principle.

Q2. What does LIFO mean?

Last In First Out, where the last inserted element is removed first.

Q3. What is Push?

Push inserts an element onto the top of the stack.

For Faculty Use

Correction Parameters	Formative Assessment [40%]	Timely completion of Practical [40%]	Attendance / Learning Attitude [20%]	
Marks Obtained				